

Object Oriented Programming Lab Record

Experiment No: 08

Title: Implementation of Operator Overloading in C++.

Objective:

To understand and implement the concept of operator overloading in C++ for performing operations on user-defined data types.

Theory:

Operator overloading is a feature in C++ that allows redefining the behavior of operators for user-defined types (class). It enables the same operator to have different meanings based on the operands.

Syntax:

```
return_type operator op (argument_list);
```

Types of Operators That Can Be Overloaded:

- Arithmetic Operators (+, -, *, /)
- Relational Operators (==, !=, >, <)
- Unary Operators (++ and --)
- Input/Output Operators (<< and >>)

Operators That Cannot Be Overloaded:

- Scope Resolution (::)
- sizeof
- Member Access (.)
- Pointer to Member Access (.*)
- Ternary Operator (?:)

Program:

```
#include <iostream>
```

```
using namespace std;
```

```
class Complex {
```

```
    float real;
```

```
    float imag;
```

```
public:
```

```
    Complex() {
```

```
        real = 0;
```

```
        imag = 0;
```

```
    }
```

```
    Complex(float r, float i) {
```

```
        real = r;
```

```
        imag = i;
```

```
    }
```

```
    Complex operator + (Complex obj) {
```

```
        Complex temp;
```

```
        temp.real = real + obj.real;
```

```
        temp.imag = imag + obj.imag;
```

```
        return temp;
```

```
    }
```

```
    void display() {
```

```

        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(2.5, 3.5);
    Complex c2(1.5, 4.5);

    Complex c3 = c1 + c2;

    cout << "First Complex Number: ";
    c1.display();
    cout << "Second Complex Number: ";
    c2.display();
    cout << "Sum of Complex Numbers: ";
    c3.display();

    return 0;
}

```

Output:

First Complex Number: 2.5 + 3.5i
 Second Complex Number: 1.5 + 4.5i
 Sum of Complex Numbers: 4 + 8i

Result:

The program successfully demonstrates operator overloading using the '+' operator to add two complex numbers.

Viva Questions and Answers:

1. Q: What is operator overloading in C++?

A: Operator overloading allows redefining the behavior of operators for user-defined types like classes.

2. Q: Which keyword is used for operator overloading?

A: The 'operator' keyword is used before the operator symbol.

3. Q: Can we overload all operators in C++?

A: No, some operators like ::, sizeof, ., .*, and ?: cannot be overloaded.

4. Q: What is the difference between operator overloading and function overloading?

A: Operator overloading changes operator behavior, while function overloading allows multiple functions with the same name.

5. Q: What is the purpose of operator overloading?

A: It allows performing operations on objects in a natural and intuitive way similar to built-in data types.